All exercises are solved by programming in **Matlab** (at least version 2020b is required). Make sure that you are using the right version of Matlab.

In case you encounter syntax errors in your code, first refer to the Matlab documentation before asking the teaching assistants for help.

Point Cloud Segmentation and Object Detection

This exercise shows how to process 3-D lidar data from a sensor mounted on a vehicle by segmenting the ground plane and finding nearby obstacles. This can facilitate drivable path planning for vehicle navigation. The exercise also shows how to visualize streaming lidar data. It is based on the Automated Driving as well as Lidar Toolbox from Matlab.

Current prototype vehicles for automated driving are equipped with 3-D lidar sensors due to their superior measurement accuracy compared to radar and camera sensors. The measured point cloud of a single lidar scan can be defined as a set $P = \{\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \ldots, \mathbf{p}_K\}$ of points $\mathbf{p}_k$. Figure 1 shows an example of a recorded point cloud. Next to the 3-D coordinates, a single point $\mathbf{p}_k$ may contain additional measured / estimated attributes like for example the intensity i of the reflected ray, i.e. $\mathbf{p}_k = [x_k \; y_k \; z_k \; i_k \; \cdots]^T$

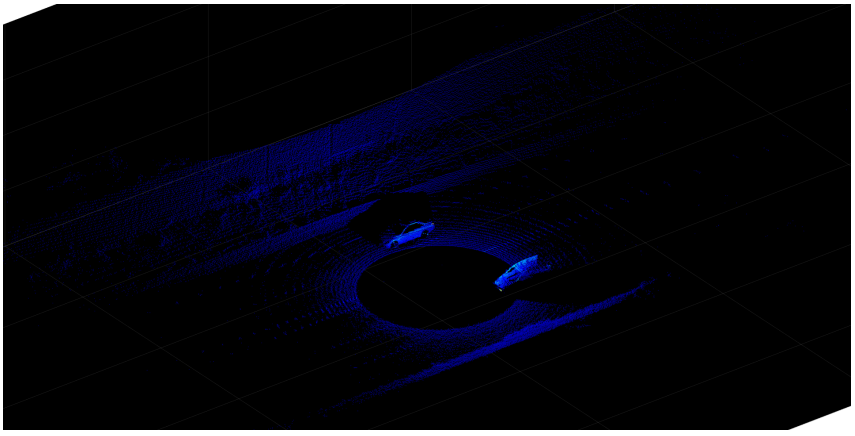

Figure 1: Depiction of a point cloud of a full 3-D lidar scan on a highway.

The lidar data used in this exercise was recorded using an Ouster OS1-64 sensor mounted on the test vehicle of the Institute of Control Theory and Systems Engineering (RST) (see figure 2).

Every recorded scan has been converted to the Matlab data structure `pointCloud` as an efficient and organized storing format of the point cloud. Efficiently processing this vast amount of data using spatial indexing is key to the performance of the sensor processing pipeline. This efficiency is achieved using the `pointCloud` object, which internally organizes the data using a K-d tree data structure [2].

The `Location` property of the `pointCloud` is an M-by-N-by-3 matrix, containing the xyz coordinates of points in meters. Here, M is the number of scan layers (64 for the OS1-

Figure 2: Image of the test vehicle of the RST. A Nissan Leaf ZE0 equipped with in total six FLIR Chameleon® 3 cameras, an Ouster OS1-64 3-D lidar and a ADMA-G RTK-GPS system.

64) whereas N corresponds to the number of azimuth angle bins (2048 for the chosen measurement mode of the test drive).

Visualization of Point Cloud Data

In order to visualize the point cloud data, the `pcplayer` can be used.

```
lidarViewer = pcplayer(xlimits, ylimits, zlimits)
```

Sets up the region around the lidar sensor specified by `xlimits`, `ylimits` and `zlimits` to display the corresponding measurement data. The point cloud `ptCloud` can be displayed using:

```
view(lidarViewer, ptCloud)
```

1) Download the *pointClouds_highway.mat* file from the Moodle workspace and load it into Matlab using the `load` command. The cell array `pointClouds` is loaded into the Matlab workspace where every cell element corresponds to an organized lidar scan in form of a `pointCloud` object.

2) Visualize the lidar scans contained in `pointClouds` using the `pcplayer`. Set appropriate limits in x-direction, y-direction and z-direction. Use the command `pause(s)` with an appropriate time delay of `s` seconds to visualize the data in slower motion.

3) Inspect the point clouds of the test drive. What kind of objects can you recognize?

Point Cloud Segmentation and Object Detection

In this exercise, we will be segmenting points belonging to the ground plane and nearby obstacles. Subsequently, 3D bounding boxes will be fitted to the obstacle points representing other detected vehicles. The pipeline is summarized in figure 3.

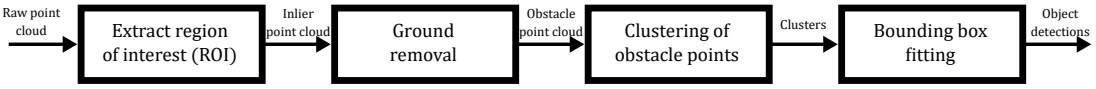

Figure 3: Point cloud segmentation pipeline according to [1].

Generally, every module of the pipeline needs to find the indices of the points corresponding to the respected class (e.g. region of interest, ground, etc.). If the `indices` have been found, the corresponding point set can be selected using

```
ptCloudOut = select(ptCloud, indices, 'OutputSize', 'full')
```

Here, `indices` may be a logical or integer index array. Specify the `'OutputSize'` as `'full'` to retain the organized structure of the point cloud.

For identifying points that belong to the ground, the function

```
[~,inlierIndices,outlierIndices] = pcfitplane(ptCloudIn,maxDistance,referenceVector
  ,maxAngularDistance);
```

may be used. This function implements a M-estimator sample consensus (MSAC) [4] based algorithm for robustly finding a plane within the data points contained in the point cloud. The method is outlined in algorithm 1. Next to the distance threshold T (`maxDistance`), the algorithm additionally takes two further arguments `referenceVector` and `maxAngularDistance`. Given these arguments, the normal vector of the fitted plane may only deviate `maxAngularDistance` degrees from the`referenceVector`. The output is an array of indices for the points identified as belonging to the ground (`inlierIndices`) as well as obstacles (`outlierIndices`).

Algorithm 1 Algorithm for MSAC based plane fit to [4].

1: **procedure** FitPlane(P, T)
2: Randomly select 3 points from P.
3: Compute plane model parameters given the selected points.
4: Compute distances e_i of all points to estimated plane.
5: Determine costs according to $\sum_i \rho(e_i^2)$, where $\rho(e^2) = \begin{cases} e^2, & e < T^2 \\ T^2, & e \geq T^2 \end{cases}$.
6: Repeat steps 2 to 5 until maximum number of iterations reached.
7: Choose point set with lowest costs.
8: **end procedure**

To segment objects within the obstacle point cloud the function

```
[labels,numClusters] = pcsegdist(ptCloudIn,minDistance);
```

can be used. This function implements Euclidean distance based clustering as described in [3]. Pseudo-code of this method is given in algorithm 2.

Algorithm 2 Algorithm for Euclidean distance clustering according to [3].

procedure EUCLIDEANDISTANCECLUSTERING(P, d_{th})
  $C \leftarrow \emptyset$, $Q \leftarrow \emptyset$
  for $\mathbf{p}_i \in P$ **do**
    Q.add($\mathbf{p}_i$)
    for $\mathbf{p}_i \in Q$ **do**
      $P_i^k \leftarrow$ FindNeighbors($\mathbf{p}_i$, d_{th})
      for $\mathbf{p}_i^k \in P_i^k$ **do**
        if $\mathbf{p}_i^k$ not processed **then**
          Q.add($\mathbf{p}_i^k$)
        end if
      end for
    end for
    C.add(Q)
    $Q \leftarrow \emptyset$
  end for
  return C
end procedure

Finally, a bounding box can be fitted to the segmented cluster to estimate the dimensions of the detected object. Regarding point clusters of vehicles, an L-shape-based fitting algorithm yields accurate results. The Matlab function `pcfitcuboid`

```
model = pcfitcuboid(ptCloudIn,indices)
```

implements the algorithm of [5] and returns a `cuboidModel` which specifies the 3D bounding box by its center point, dimensions (length, width, height) as well as orientation (roll, pitch, yaw). Here, `indices` corresponds to the indices of the corresponding point cluster to fit the bounding box to.

The goal of L-shape fitting is to find a bounding box (2D rectangle) that fits best to the cluster points. One classical criterion in evaluating the fitting performance is least squares which involves the following optimization problem:

$$\begin{aligned}
&\underset{P,\theta,c_x,c_y}{\text{minimize}} \sum_{i\in P_x} (x_i \cos\theta + y_i \sin\theta - c_x)^2 + \sum_{j\in P_y} (-x_j \sin\theta + y_j \cos\theta - c_y)^2 \\
&\text{subject to } P_x \cup P_y = P \in \mathbb{R}^{x\times 2} \\
&\qquad\qquad c_x, c_y \in \mathbb{R},\ \theta \in [0^\circ, 90^\circ)
\end{aligned}$$

Here, the squared error is minimized by looking for the optimal disjunction P_x and P_y which split the clustered points into two sets as well as the optimal parameters (θ, c_x, c_y) for the two perpendicular lines which correspond to the points in P_x and P_y respectively. Due to the combinatorial complexity, it is impractical to solve the above optimization problem in real-time. The authors of [5] therefore propose a grid-search based for $\theta \in [0°, 90°)$ to find the best-fit bounding box approximately. Pseudo-code of the method is outlined in algorithm 3.

Algorithm 3 Grid-search-based algorithm for L-shape-based bounding box fitting based on [5].

procedure GridSearchBasedLShapeFitting(P)
  $J \leftarrow \emptyset$
  for $\theta \in [0°, 90°)$ **step** δ **do**
    $\hat{e}_x \leftarrow [\cos\theta, \sin\theta]^\intercal$, $\hat{e}_y \leftarrow [-\sin\theta, \cos\theta]^\intercal$
    $C_x \leftarrow P \cdot \hat{e}_x$, $C_y \leftarrow P \cdot \hat{e}_y$
    $q \leftarrow$CalculateVariance(C_x,C_y)
    J.add(q)
  end for
  $\theta^* = \underset{\theta}{\operatorname{argmax}}\ J(\theta)$
  $\hat{e}_x^* \leftarrow [\cos\theta^*, \sin\theta^*]^\intercal$, $\hat{e}_y^* \leftarrow [-\sin\theta^*, \cos\theta^*]^\intercal$
  $C_x^* \leftarrow P \cdot \hat{e}_x^*$, $C_y^* \leftarrow P \cdot \hat{e}_y^*$
  $c_{x,\min}^* \leftarrow \min\{C_x^*\}$, $c_{x,\max}^* \leftarrow \max\{C_x^*\}$
  $c_{y,\min}^* \leftarrow \min\{C_y^*\}$, $c_{y,\max}^* \leftarrow \max\{C_y^*\}$
  $l^* \leftarrow c_{x,\max}^* - c_{x,\min}^*$, $w^* \leftarrow c_{y,\max}^* - c_{y,\min}^*$
  $x_c^* \leftarrow (c_{x,\min}^* + \frac{l^*}{2})\cos\theta^*$, $y_c^* \leftarrow (c_{y,\min}^* + \frac{w^*}{2})\sin\theta^*$
  return $(x_c^*, y_c^*, \theta^*, l^*, w^*)$
end procedure
procedure CalculateVariance(C_x,C_y)
  $c_{x,\min} \leftarrow \min\{C_x\}$, $c_{x,\max} \leftarrow \max\{C_x\}$
  $c_{y,\min} \leftarrow \min\{C_y\}$, $c_{y,\max} \leftarrow \max\{C_y\}$
  $D_x \leftarrow \min\{|c_{x,\max} - C_x|, |C_x - c_{x,\min}|\}$
  $D_y \leftarrow \min\{|c_{y,\max} - C_y|, |C_y - c_{y,\min}|\}$
  $E_x \leftarrow \{D_x | D_x < D_y\}$, $E_y \leftarrow \{D_y | D_y < D_x\}$
  return $-\mathrm{Var}(E_x) - \mathrm{Var}(E_y)$
end procedure

To visualize the intermediate steps of the segmentation pipeline, a helper class `lidarViewUpdater` is provided in the Moodle workspace. The class is initialized with the handle of the `pcplayer`, i.e

```
viewUpdater = lidarViewUpdater(lidarViewer);
```

The class provides the method

```
updateView(viewUpdater, ptCloud, labelIndices)
```

to visualize the point cloud `ptCloud`. Here, `labelIndices` is a struct with the fields `ROI`, `Ground`, `Obstacle` and `Cluster` where every field contains a $M \times N$ logical index array indicating if the corresponding point is assigned to the class defined by the field name. The method `updateView` can additionally take a fourth argument `detections` which is a cell array of `cuboidModel` to visualize the detected objects with 3D bounding boxes.

To analyze the behavior of the different pipeline modules as well as for tuning the parameters of the corresponding methods, we will first only consider a single point cloud scan in the following task, e.g.

```
ptCloud = pointClouds{150};
```

4) Implement a function

```
[ptCloudOut,indices,croppedIndices] = cropPointCloud(ptCloudIn,xLim,yLim,
zLim)
```

to extract a region of interest specified from the point cloud where `xLim=[xLowerLimit,xUpperLimit]`, etc. For this logical indexing of the `Location` property of the `ptCloudIn` shall be used. The point cloud `ptCloudOut` should contain only the points included in the ROI while retaining the organized point cloud format (i.e. using `select(ptCloud, indices, 'OutputSize', 'full')`). Assign the determined `indices` of the ROI to `labelIndices.ROI`.

5) Visualize the ROI using the `updateView` method. Choose reasonable values for `xLim`,`yLim`,`zLim`. The results should look similar to figure 4

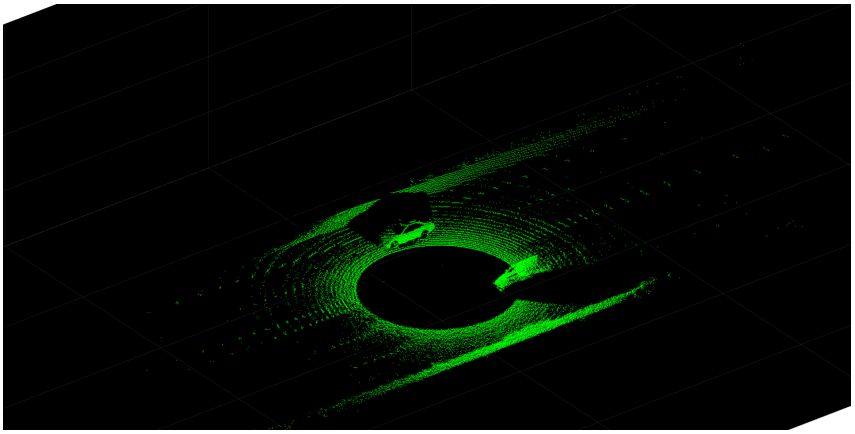

Figure 4: Segmented region of interest (ROI) of the point cloud.

6) Implement a function

```
[ptCloudOut,obstacleIndices,groundIndices] = removeGroundPlane(ptCloudIn,
maxZHeight,maxGroundDist,referenceVector,maxAngularDist,currentIndices)
```

to remove the ground points from the cropped point cloud by using the Matlab function `pcfitplane`. To enhance the fitting of the ground plane, a z-region of the point cloud defined by `maxZHeight` should be pre-selected before applying `pcfitplane`. The array `currentIndices` corresponds to the indices determined by the `cropPointCloud` method and is needed to compute `obstacleIndices` and `groundIndices`. Assign the ground indices to `labelIndices.Ground` and the obstacle indices to `labelIndices.Obstacle`.

7) Visualize the segmented ground and obstacles using the `updateView` method. Tune the parameters `maxZHeight`,`maxGroundDist`,`referenceVector` and `maxAngularDist` to obtain reasonable results. The results should look similar to figure 5.

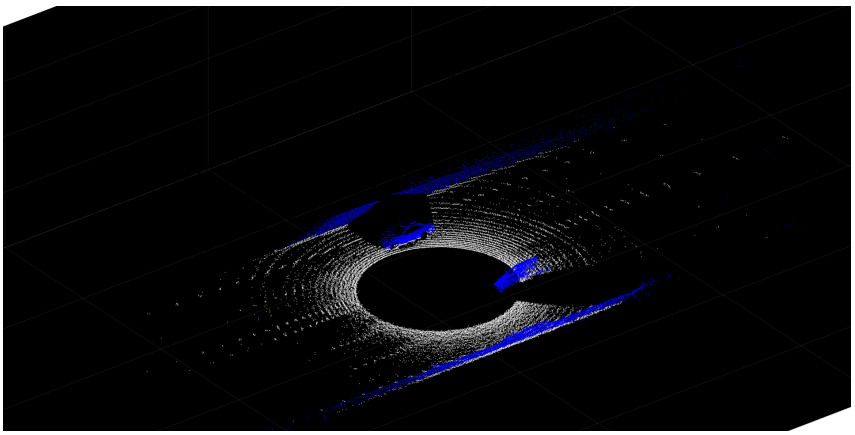

Figure 5: Segmented ground and obstacle points.

8) Next, segment obstacle point into clusters by implementing the function

```
[clusterIndices] = segmentObstaclePointsIntoClusters(ptCloudIn,minDistance,
minDetsPerCluster)
```

The function should be based on the `pcsegdist` method. Additionally, the function should only output clusters that have at least `minDetsPerCluster` points to remove clutter detections. The output `clusterIndices` should be a cell array of indices. Assign it to `labelIndices.Cluster`.

9) Visualize the determined point clusters using the `updateView` method. Tune the parameters `minDistance` and `minDetsPerCluster` to obtain reasonable results. The results should look similar to figure 6.

10) Implement a function

```
[detections] = getBoundingBoxDetections(ptCloud,clusterIndices,maxLength,
maxWidth)
```

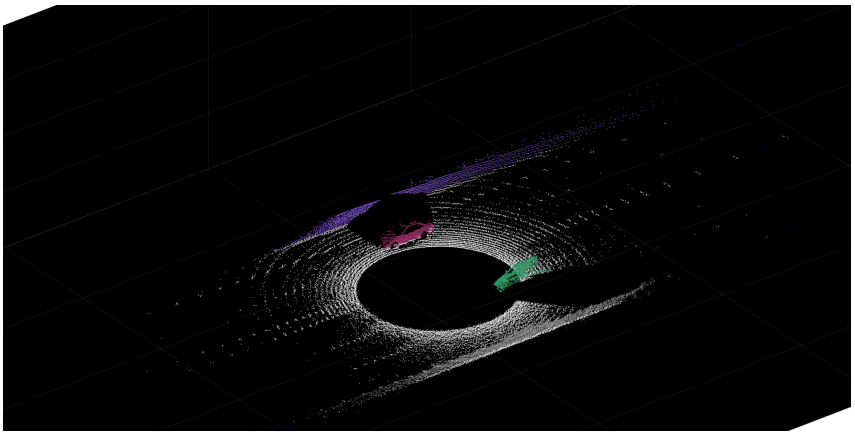

Figure 6: Segmented ground and obstacle points.

that uses the Matlab function `pcfitcuboid` to fit bounding boxes to the segmented clusters. The output `detections` should be a cell array of `cuboidModel` representing the fitted bounding boxes. Additionally, only detected objects with a maximum length and width of `maxLength` and `maxWidth` respectively shall be outputted.

11) Visualize the fitted bounding boxes using the `updateView` method by providing `detections` as an additional argument. Tune the parameters `maxLength` and `maxWidth` to obtain reasonable results. The results should look similar to figure 7.

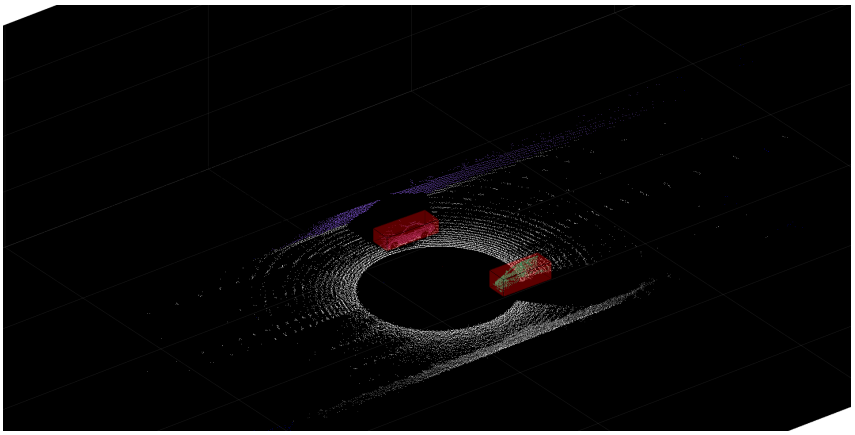

Figure 7: Segmented ground and obstacle points.

12) Now that the point cloud processing pipeline for a single lidar scan has been performed, put everything together to process the sequence of the complete recorded test drive using a `for`-loop.

Literaturverzeichnis

[1] Arya Arya Senna Abdul Rachman. 3d-lidar multi object tracking for autonomous driving: Multi-target detection and tracking under urban road uncertainties. Master's thesis, TU Delft, 2017.

[2] Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.

[3] Radu Bogdan Rusu. *Semantic 3D Object Maps for Everyday Manipulation in Human Living Environments*. PhD thesis, Computer Science department, Technische Universitaet Muenchen, Germany, 2009.

[4] Philip HS Torr and Andrew Zisserman. Mlesac: A new robust estimator with application to estimating image geometry. *Computer vision and image understanding*, 78(1):138–156, 2000.

[5] Xiao Zhang, Wenda Xu, Chiyu Dong, and John M Dolan. Efficient l-shape fitting for vehicle detection using laser scanners. In *2017 IEEE Intelligent Vehicles Symposium (IV)*, pages 54–59. IEEE, 2017.